EEL 6878: Modeling and Artificial Intelligence

Spring 2026

Instructor: Tong Wu

Homework: Graph Signal Processing & Graph Neural Networks

Contents

Graph Signal Processing

In this section, you will use the existing package `pygsp`[1] to complete tasks related to graph signal processing.

Question 1. Spectral Analysis of Graph Signals

The spectral content of a signal indicates whether it is low-pass, band-pass, or high-pass. Intuition transfers from classical Fourier analysis to the graph setting.

1. Build a sensor graph using:

```
G = pygsp.graphs.Sensor(N=100, seed=1)
```

2. Compute its Graph Fourier basis.
3. Create two kinds of graph signals by following these steps:
   (a) Generate random signals of size 1 for each node in the graph according to the standard normal distribution.
   (b) Generate two different filter banks of Heat kernels[2] characterized by their respective parameters: `scale` $= 0$ for the first filter and `scale` $= 10$ for the second filter. These parameters alter the filter's behavior, leading to different spectral characteristics of the processed signals.
   (c) Apply the defined filters to the randomly generated signals. This process will create two distinct types of graph signals: one demonstrating low-pass characteristics and the other high-pass characteristics.
4. Calculate the Graph Fourier Transform (GFT) for both the low-pass and high-pass graph signals to obtain the signals in the Graph Fourier domain $\hat{\boldsymbol{x}}$.
5. Plot the graph spectral content of $\hat{\boldsymbol{x}}$, i.e., $|\hat{\boldsymbol{x}}|$, for both signals.
6. Evaluate the smoothness of the two graph signals, defined as:

$$S(\boldsymbol{x}) = \boldsymbol{x}^\top \mathbf{L}\, \boldsymbol{x}$$

   where $\mathbf{L}$ is the graph Laplacian. Compare and discuss the smoothness values of the two signals.
7. Is the graph signal generated by `scale` $= 10$ a high-pass or a low-pass graph signal? Justify your answer.

Question 2. Synthetic and Real-World Networks

1. Generate (or load) the following *undirected* graphs and plot them:
   (a) An Erdős-Rényi graph with $N = 50$ and $p = 0.1$ using `pygsp`.
   (b) A Ring graph with $N = 50$ using `pygsp`.
   (c) The Cora[3] citation network from PyTorch Geometric.[4]
2. Since we are considering undirected graphs, the Laplacian matrix is real symmetric and thus admits an eigen-decomposition:

$$\mathbf{L} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top,$$

   where $\mathbf{U} = (\mathbf{u}_1, \ldots, \mathbf{u}_N) \in \mathbb{R}^{N\times N}$ is the matrix of orthonormal eigenvectors and $\boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_N)$ with sorted eigenvalues:

$$\lambda_1 \le \lambda_2 \le \cdots \le \lambda_N.$$

[1] https://pygsp.readthedocs.io/en/stable/
[2] https://pygsp.readthedocs.io/en/latest/reference/filters.html
[3] The Cora dataset consists of 2,708 scientific publications classified into one of seven classes and 5,429 citation links. Each publication is described by a 0/1-valued word vector of length 1,433 indicating word presence.
[4] https://pytorch-geometric.readthedocs.io/en/latest/modules/datasets.html

- Plot the graph frequencies (eigenvalues) of the Erdős-Rényi, Ring, and Cora graphs, from smallest to largest.
- Plot the eigenvectors $\mathbf{u}_2$, $\mathbf{u}_3$, and $\mathbf{u}_6$ overlaid on the Cora graph using a node colormap.[5]

3. Compute the covariance matrix of the Cora node feature data. Use t-SNE to obtain two principal components for visualization, then apply K-means clustering with $K = 7$ classes. Report:
   - The Adjusted Rand Index (ARI) of the resulting labels.
   - The Silhouette score of the resulting labels.
4. Let's define a low-pass filter in the spectral domain:

$$\tilde{h}(\lambda) = \frac{1}{1 + \alpha\lambda}.$$

Given a noisy version $\boldsymbol{x}_{\text{noisy}}$ of a smooth signal, one can denoise it as:

$$\boldsymbol{x}_{\text{denoised}} = \mathbf{U}\,\tilde{h}(\boldsymbol{\Lambda})\,\mathbf{U}^\top\,\boldsymbol{x}_{\text{noisy}}.$$

(a) Consider the Ring graph from Part (a) and define the following graph signal:

$$\boldsymbol{x} = \sin(i) + \cos(2i) + 0.5\cos(30i),$$

where $i = \texttt{np.linspace}(-\pi, \pi, N)$. Add white noise drawn from a Uniform distribution on $[-0.4, 0.4]$ to the signal.

(b) Plot the noisy graph signal.

(c) Compute the signal in the Graph Fourier domain $\hat{\boldsymbol{x}}$ and plot its spectral content $|\hat{\boldsymbol{x}}|$.

(d) Define the low-pass filter with $\alpha = 2$ using:

```
g = pygsp.filters.Filter(G, h)
```

(e) Apply g to denoise the signal:

```
x_denoised = g.filter(x, method='exact')
```

(f) Plot the denoised graph signal and its graph spectral content. Compare with the noisy version and discuss the effect of the low-pass filter.

[5] See https://networkx.org/documentation/stable/auto_examples/drawing/plot_node_colormap.html for guidance.

Graph Neural Networks

Question 3. Node Classification with Graph Neural Networks

In this problem, you will use PyTorch Geometric to build graph neural networks for node classification on the Cora dataset.[6]

1. **Model Architecture.** Build a two-layer Chebyshev Graph Neural Network (ChebNet) using PyTorch Geometric's `ChebConv` operator with the following specifications:
   - **Layer 1:** Input channels = number of node features in the dataset; output channels = 40; Chebyshev order $K = 5$.
   - **Layer 2:** Input channels = 40; output channels = number of label classes in the dataset.
   - **Activation functions:** Use `relu` after the first layer and `log_softmax` at the output layer to produce predicted class probabilities.
2. **Training.** Train your GNN using the following setup:
   - Optimizer: `Adam` with learning rate 0.01 and weight decay 5×10^{-4}.[7]
   - Loss function: `torch.nn.functional.nll_loss` (negative log-likelihood loss), or any other suitable loss for your model.
   - Plot the training loss as a function of the epoch number.
3. **Evaluation and Visualization.** Test your trained GNN and do the following:
   - Report the classification accuracy on the test set.
   - Visualize the Cora graph with node colors indicating (i) ground truth labels and (ii) predicted labels. Discuss any notable differences.

[6] A tutorial on GNN implementation is available at https://pytorch-geometric.readthedocs.io/en/latest/notes/introduction.html.

[7] You may adjust the learning rate and weight decay as needed for your model.

References

[1] N. Perraudin et al., "GSPBOX: A toolbox for signal processing on graphs," *arXiv preprint arXiv:1408.5781*, 2014. https://pygsp.readthedocs.io/en/stable/

[2] A. K. McCallum, K. Nigam, J. Rennie, and K. Seymore, "Automating the Construction of Internet Portals with Machine Learning," *Information Retrieval*, vol. 3, pp. 127–163, 2000.

[3] M. Fey and J. E. Lenssen, "Fast Graph Representation Learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.